FLASK

Agenda

- **Session 1:** 1) Introduction to Flask
  2) What is Flask?
  3) What is Web Framework?
  4) Flask's Role in Web Development
  5) Why Choose Flask?
  5) Features of Flask
- **Session 2**: 1) Installation
  2) Routing
- **Session 3:** 1) Variable’s Rule
  2) URL Building
  3) HTTP Methods
- **Session 4:** 1) Templates
  2) Forms
- **Session 5**: 1) Static Files
  2) Advantages

Introduction to Flask

- Flask is a lightweight and versatile web framework for Python.
- It allows developers to build web applications quickly and with minimal boilerplate code.

What is Flask?

- Flask is a micro web framework written in Python.
- **Armin Ronacher,** who leads an international group of Python enthusiasts named Pocco, develops it.
- It is based on the Werkzeug WSGI toolkit and Jinja2 template engine. Both are Pocco projects.
- Flask is designed to be simple and easy to use, while still providing flexibility and extensibility.

What is Web Framework?

Web Application Framework or simply Web Framework represents a collection of libraries and modules that enables a web application developer to write applications without having to bother about low-level details such as protocols, thread management etc.

WSGI

Web Server Gateway Interface (WSGI) has been adopted as a standard for Python web application development. WSGI is a specification for a universal interface between the web server and the web applications.

Jinja2

Jinja2 is a popular templating engine for Python. A web templating system combines a template with a certain data source to render dynamic web pages.

Flask's Role in Web Development

- Flask facilitates the creation of web applications by providing tools and utilities for handling HTTP requests, routing URLs, rendering templates, managing sessions, and more.
- It serves as the foundation for building dynamic and interactive web applications, APIs, and microservices.

Why Choose Flask?

- Flask is lightweight and unopinionated, allowing developers to make decisions based on their specific project requirements.
- It has a minimalistic design, making it easy to learn and use, especially for beginners.
- Flask provides a modular approach to building web applications. Instead of including every possible feature directly into the Flask core framework, Flask keeps its core lightweight and minimalistic.

Features of Flask

- Minimalistic and Lightweight
- Built-in Development Server
- Extensive Documentation
- Jinja2 Templating Engine
- Support for RESTful APIs
- Flask-WTF for Form Handling
- Secure Cookies and Sessions
- URL Routing
- Flask Extensions Ecosystem

Installation

Installation: pip install flask

Creating a Basic Flask App

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, Flask!'

if __name__ == '__main__':
    app.run(debug=True)
```

app.route(rule, options)

- The **rule** parameter represents URL binding with the function.
- The **options** is a list of parameters to be forwarded to the underlying Rule object.

app.run(host, port, debug, options)

- Modern web frameworks use the routing technique to help a user remember application URLs. It is useful to access the desired page directly without having to navigate from the home page.

The **route()** decorator in Flask is used to bind URL to a function.

```
@app.route('/hello')
def  hello_world():
  return 'hello world'
```

Http protocol is the foundation of data communication in world wide web. Different methods of data retrieval from specified URL are defined in this protocol.

Routing

URL Routing in Flask

- URL routing is the process of mapping URLs to view functions in a web application.
- Flask uses the `@app.route()` decorator to define routes for different URLs.

Decorators (`@app.route()`)

- Decorators are used to associate a URL with a view function in Flask.
- The `@app.route()` decorator specifies the URL pattern and the function to execute when that URL is accessed.

Routing with Variables

- Flask supports dynamic URL routing by using variable parts in the URL pattern.
- Variable parts are enclosed in `< >` brackets in the URL pattern, and their values are passed as arguments to the view function.

Handling HTTP Methods (GET, POST, etc.)

- Flask routes can handle different HTTP methods (e.g., GET, POST, PUT, DELETE) by specifying them as arguments to the `methods` parameter of the `@app.route()` decorator.
- Each route can have multiple methods associated with it, allowing for different actions based on the HTTP request method.

Variable’s Rule

- It is possible to build a URL dynamically, by adding variable parts to the rule parameter.
- This variable part is marked as **<variable-name>**.
- It is passed as a keyword argument to the function associated with the rule.

  1) int - accept integer
  2) float - for floating point value
  3) path - accept slashes used for directory separator character

```
from flask import Flask
app=Flask(__name__)

@app.route('/')
def welcome():
   return “Congratulations to all”

@app.route(“/success/<int:score>”)
def success(score):
   return “He/She is passed and mark is” + str(score)

@app.route(“/fail/<int:score>”)
def fail(score):
   return “He/She is failed and mark is” + str(score)

if __name__ == '__main__':
  app.run(debug = True)
```

URL Building

- The url_for() function is very useful for dynamically building a URL for a specific function.
- The function accepts the name of a function as first argument, and one or more keyword arguments, each corresponding to the variable part of URL.

```
from flask import Flask, redirect, url_for
app=Flask(__name__)

@app.route('/')
def welcome():
   return “Congratulations to all”

@app.route(“/success/<int:score>”)
def success(score):
   return “He/She is passed and mark is” + str(score)

@app.route(“/fail/<int:score>”)
def fail(score):
   return “He/She is failed and mark is” + str(score)
```

```
### Result checker
@app.route('/results/<int:marks>')
def results(marks):
   result = “ ”
   if marks<50:
      result=”fail”
   else:
      result=”success”
   return redirect(url_for(result, marks=score))

if __name__ == '__main__':
  app.run(debug = True)
```

The following table summarizes different http methods –

Sr.No.	Methods & Description
1	GET Sends data in unencrypted form to the server. Most common method.
2	HEAD Same as GET, but without response body
3	POST Used to send HTML form data to server. Data received by POST method is not cached by server.
4	PUT Replaces all current representations of the target resource with the uploaded content.

Templates

Jinja2 Templating Engine

- Jinja2 is a popular templating engine for Python and Flask.
- It allows for creating dynamic HTML templates by embedding Python-like syntax within HTML files.

Creating and Rendering Templates

- Templates are HTML files containing placeholders and control structures.
- Flask uses the `render_template()` function to render templates and pass data to them.

Template Inheritance

- Template inheritance allows for creating a base template with common elements shared across multiple pages.
- Child templates extend the base template and override specific sections as needed.

Passing Data to Templates

- Data can be passed from Flask views to templates using template variables.
- Template variables are accessible within HTML using double curly braces `{{ }}`.

Using Template Filters

- Jinja2 provides filters for modifying template variables before they are displayed.
- Filters are applied using the pipe `|` character followed by the filter name.

Forms

Flask-WTF Extension

- Flask-WTF is a Flask extension that simplifies form handling by providing utilities for form creation, validation, and CSRF protection.

Form Handling in Flask

- Forms play a crucial role in web applications for capturing user input, such as login credentials, registration details, or any other data.
- Flask-WTF extends Flask by providing a set of classes and functions to streamline form handling.

Creating and Validating Forms

- Flask-WTF allows developers to define forms as classes, where each form field is represented as an attribute.
- Form validation is performed automatically, ensuring that the data submitted by users meets specified criteria.

- Example:

```
from flask_wtf import FlaskForm
from wtforms import StringField, SubmitField
from wtforms.validators import DataRequired, Length

class MyForm(FlaskForm):
    name = StringField('Name', validators=[DataRequired(), Length(min=2, max=20)])
    submit = SubmitField('Submit')
```

Forms

CSRF Protection

- Cross-Site Request Forgery (CSRF) attacks exploit the trust between a user and a website to perform malicious actions.
- Flask-WTF provides built-in CSRF protection to mitigate such attacks by generating and validating CSRF tokens for each form.

Integrating Forms with Views

- Forms are typically rendered within view functions (routes) and displayed in HTML templates using Flask's `render_template` function.
- Views handle form submission, validate input, and process data before redirecting or rendering a response.
- Example:

```
@app.route('/submit', methods=['GET', 'POST'])
def submit_form():
    form = MyForm()
    if form.validate_on_submit():
        # Process form data
        name = form.name.data
        return f'Success! Your name is {name}.'
    return render_template('form.html', form=form)
```

Static Files

Serving Static Files (CSS, JavaScript, Images):
- Flask allows you to serve static files such as CSS, JavaScript, and images directly from the server.
- These files are typically stored in a directory named `static` in your Flask application's root directory.

Static Folder Convention:
- The `static` folder serves as the convention for storing static files in Flask applications.
- Flask automatically recognizes this folder and serves its contents as static files.

Linking to Static Files in Templates:
- To link to static files in HTML templates, use the `url_for()` function provided by Flask.

- Example:

```
<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
<script src="{{ url_for('static', filename='script.js') }}"></script>
<img src="{{ url_for('static', filename='logo.png') }}" alt="Logo">
```

Advantages

1. **Lightweight and Minimalistic**: Flask is lightweight and has minimal dependencies, making it easy to learn and use for both small and large projects.

2. **Flexibility**: Flask provides the flexibility to choose components as needed, allowing developers to create custom solutions tailored to their specific requirements.

3. **Extensible**: Flask's modular design and extensive ecosystem of extensions enable developers to add functionality easily, such as database integration, authentication, and RESTful APIs.

4. **Easy to Get Started**: Flask has a simple and intuitive syntax, making it easy for beginners to get started with web development in Python.

5. **Built-in Development Server**: Flask comes with a built-in development server, making it convenient for testing and debugging applications locally.

Advantages

6. **Scalability**: While Flask is lightweight, it is also scalable and can handle the demands of larger applications when properly configured.

7. **Large Community and Active Development**: Flask has a large and active community of developers who contribute to its ongoing development, providing support, resources, and extensions.

8. **Jinja2 Templating Engine**: Flask integrates with the Jinja2 templating engine, allowing developers to create dynamic and reusable HTML templates easily.

9. **RESTful Support**: Flask provides built-in support for building RESTful APIs, making it ideal for creating web services and backend systems.

10. **Documentation**: Flask has extensive and well-organized documentation, making it easy to find answers to common questions and learn new concepts.